

Relatório de Automação – Projeto Mark85

Este relatório apresenta a estrutura, execução e boas práticas aplicadas na automação de testes do projeto **Mark85**, uma aplicação web de gerenciamento de tarefas. A automação contempla testes de **front-end (UI)** e **back-end (API)** utilizando o **Robot Framework**, com apoio de bibliotecas Python e integração com banco de dados MongoDB.

1. Preparação do Ambiente de Testes

Instalação e Execução da Aplicação

- Clonar os diretórios: `mark85/api` e `mark85/web`
- Instalar dependências: `npm install` em ambos os diretórios
- Iniciar servidores:
 - API: `cd api && npm run dev`
 - WebApp: `cd web && npm run dev` → acessível via `localhost:3000`
 - Encerrar execução: `Ctrl + C` no terminal

Configuração do MongoDB Atlas

- Criar um projeto (Mark85) com uma organização
- Criar usuário: `qa` / senha: `xperience`
- Adicionar IP `0.0.0.0` para acesso global (uso apenas em ambiente de aprendizado!)
- Copiar a **string de conexão** e configurar no `.env` da pasta `api`, substituindo:
 - `<password>` pela senha do usuário
 - Após `mongodb.net/`, adicionar o nome do banco: `markdb`

O MongoDB Atlas é o único componente hospedado online — a API e WebApp rodam localmente no ambiente do desenvolvedor.

2. Estrutura da Automação

Organização do Projeto

```
resources/  
├─ env.robot  
├─ base.robot  
├─ pages/  
├─ components/  
├─ services.resource  
├─ libs/database.py  
└─ fixtures/tasks.json  
tests/  
results/
```

Arquivos-chave

- `env.robot` : Armazena variáveis globais como `${BASE_URL}`
- `base.robot` : Importa todas as bibliotecas e configurações comuns
- `pages/` : Implementa o padrão Page Object para modularizar interações por tela
- `components/` : Keywords reutilizáveis para elementos comuns (alertas, cabeçalho)
- `services.resource` : Requisições REST usando `RequestsLibrary`
- `database.py` : Biblioteca customizada em Python para operações diretas no MongoDB

- `tasks.json` : Fixture para testes com dados fixos organizados por cenário

Dependências Python (requirements.txt)

```
robotframework==6.1
robotframework-browser==16.2.0
robotframework-requests==0.9.5
robotframework-jsonlibrary==0.5
robotframework-pabot==2.16.0
pymongo==4.4.0
bcrypt==4.0.1
requests==2.31.0
```

Instalação recomendada:

```
pip install -r requirements.txt
```

3. Estratégia de Testes

3.1. Cadastro de Usuário (`signup.robot`)

- Validação da aplicação online e do título da página
- Cadastro de novo usuário com massa **fixa** (dados controlados)
- Coberturas:
 - E-mail duplicado
 - Campos obrigatórios
 - E-mail inválido
 - Senhas curtas (1 a 5 dígitos, com `Test Templates`)

⚠ **FakerLibrary** foi descartada após análise de boas práticas.

Gerar dados dinâmicos repetidamente **infla o banco**, gera resíduos, e traz riscos de custo elevado em ambientes reais. A abordagem adotada foi o uso de **massa fixa com limpeza automatizada via Python** (`database.py`), alinhando a prática à sustentabilidade do ambiente de testes.

3.2. Login (`login.robot`)

- Validação de login com usuário pré-cadastrado no banco
- Verificação da saudação com nome (usando `split` para extrair primeiro nome)

3.3. Tarefas - CRUD (`tasks/`)

- Uso de fixture JSON para cada cenário:
 - `create` : Criar nova tarefa
 - `duplicate` : Evitar duplicação
 - `tags_limit` : Testar limite de tags
 - `done` : Marcar tarefa como concluída
 - `delete` : Excluir tarefa

4. Conexão com o MongoDB e Segurança

Biblioteca Python Personalizada (`database.py`)

- `Clean user from database` : Remove usuário e tarefas relacionadas
- `Insert user from database` : Adiciona novo usuário com senha **criptografada**

- Criptografia com `bcrypt`, conforme prática usada pelo desenvolvedor original
- Garante que o login funcione com o mesmo algoritmo da aplicação real

A criptografia da senha foi confirmada com o desenvolvedor, simulando com fidelidade o comportamento real da aplicação.

5. Boas Práticas Adotadas

Testes Independentes

- Cada teste é **autossuficiente**, com sua própria massa e ambiente preparado previamente
- Garante confiabilidade em execuções isoladas ou em paralelo (via `pabot`)

Evitar testes encadeados: se um falhar, os outros não devem ser impactados!

Reuso e Organização

- Modularização com Page Objects e Components
- Keywords customizadas
- Super variáveis com `Create Dictionary` para agrupamento estruturado

Integração com API e Banco

- Manipulação de dados via API (`RequestsLibrary`) e banco (`pymongo`)
- Preparação rápida do ambiente de testes

Estratégia de Massa de Testes

- JSON é usado **somente para dados complexos**
- Massa simples (como login) permanece `hardcoded`, evitando overengineering

6. Recursos Adicionais e Estratégicos

- **Screenshots automáticos:** Robot Framework gera prints automaticamente em falhas
- **Uso de CSS Selectors e XPath:** Prioriza CSS; XPath usado apenas quando necessário (ex: texto dinâmico)
- **Validadores visuais:** Checagem de estilos (ex: `line-through` em tarefas concluídas)
- **Ganchos de execução:**
 - `Suite Setup/Teardown` e `Test Setup/Teardown` configurados
- **Teste cross-browser:** Chromium por padrão, mas suporte também a Firefox e WebKit
- **Git e versionamento:**
 - `.gitignore` configurado para arquivos temporários e logs
 - Repositório pode ser replicado com `git clone` + `pip install -r requirements.txt`

Conclusão

Este projeto consolidou práticas modernas de automação com foco em:

- Independência e confiabilidade
- Modularização escalável
- Integração entre camadas (UI, API, dados)
- Sustentabilidade da massa de testes

Apesar de ser um ambiente de aprendizado, foram adotadas decisões técnicas alinhadas ao mercado profissional de QA. O uso de automação orientada a dados, testes independentes e integração backend são uma base sólida adotada em projetos maiores e reais.